TECHNICAL ARCHITECTURE / DESIGN SYSTEM / DELIVERY

NU Connect Hub

Engineering implementation blueprint

A consolidated, industry oriented product specification using React, TypeScript, Webpack, Python FastAPI, PostgreSQL and explainable student recommendations.

APPLICATION

Profiles, collaborator discovery, projects, messaging and video rooms.

BACKEND

Python FastAPI with Python Socket.IO, business logic and machine learning.

DESIGN

shadcn/ui preset b27GdDLG, responsive screens and light/dark brand themes.

DELIVERY

Auditable source migration, security gates, evaluations, tests and public demo.

IMPLEMENTATION SPECIFICATION | PROPOSED

04 November 2026 / Version 2.3

Repository: github.com/kshitijkumar-northestern/neu-connect-hub

READ THIS FIRST

Build specification and decisions

This is a planned implementation, not a claim of completed development. It replaces the previous Node.js runtime architecture. JavaScript tooling remains necessary to compile the React frontend; no Node.js application server is required.

Approved technology decisions

Frontend: React + TypeScript + Webpack 5 + shadcn/ui preset **b27GdDLG**.
Backend: Python + FastAPI + Python Socket.IO.
Persistence: PostgreSQL + SQLAlchemy + Alembic + pgvector.
Recommendation: Sentence Transformers and scikit learn inside the Python codebase.

Contents

01 Product scope and release definition 3
02 Repository audit, provenance and migration 3
03 Final system architecture 4
04 Industry standard repository structure 5
05 Design system, logo and light/dark themes 7
06 Relational database and migrations 8
07 FastAPI contracts and Python realtime 9
08 Machine learning recommendations 10
09 Machine learning evaluation and Kaggle 11
10 Security, privacy and operational trust 12
11 Testing, release automation and deployment 12
12 Environment and development workflow 13
13 Delivery phases and recruiter proof 14
14 Scan to Connect: personal QR profiles 16
15 Risks, architectural decisions and sources 18

01 Product scope and release definition

Problem and primary user journey

NU Connect Hub helps verified university students find people who share compatible goals, contribute complementary expertise and have overlapping availability. Chat and video are supporting tools; the core user value is intelligent peer discovery and practical team formation.

```
Student signs up -> verifies campus email -> completes profile
 -> discovers eligible peers or projects -> sees match reasons
 -> sends or accepts an invitation (or scans a QR profile code)
 -> chats -> joins a room
```

First release: intentionally narrow

Feature	Required user behavior
Profiles	Verified students control interests, skills, goals, weekly availability and visibility.
Discover	See up to five ranked eligible peers with evidence based explanations.
Teams	Publish a project, identify missing skills, review applicants and manage capacity.
Connect	Request a collaboration, accept or decline, send durable messages.
Scan to Connect	Scan a personal QR code to preview a profile and send a permissioned connection request.
Rooms	Join authorized WebRTC rooms with microphone and camera controls.
Trust	Block, report, opt out of discovery and delete a profile and derived data.

Nonfunctional engineering targets, not measured outcomes

- Protected operations reject unauthorized users at both HTTP and Socket.IO event boundaries.
- Repeated message sends cannot create duplicate persisted messages; reconnecting clients recover missed messages.
- Benchmark ranking with 100 eligible candidates and target a warm 95th percentile under 750 milliseconds, then report measurements.
- Keyboard operability, correct focus behavior, readable contrast and responsive layouts in both themes.
- A new developer can run the documented local stack and a reviewer can reproduce the machine learning evaluation.

02 Repository audit, provenance and migration

Repository	Existing state	Decision
neu-connect-hub	Destination contains a minimal README.	Create the new product here.
NUConnects	Static Bootstrap screens, onboarding concepts and site copy.	Reuse design ideas, not duplicate accounts or admin placeholders.
ConnectRoom	React and Express, MongoDB, Socket.IO events and WebRTC signaling.	Review rights; port authorized behavior and UI to new architecture.

Why this is a rewrite of the backend, not a direct merge

The previous backend uses Express, Mongoose and Socket.IO for Node.js. Those server files must be ported to FastAPI, SQLAlchemy and Python Socket.IO. Existing React components can be selectively migrated, but integration contracts and authentication flows must change. The browser WebRTC media APIs can remain.

- Do not carry over any existing `server/.env`; it is tracked in the legacy repository. Rotate any active credentials found and remove exposed history when appropriate.
- Fix invitation authorization: only the addressed recipient may accept a pending request.
- Fix inconsistent conversation fields and history querying; introduce proper relational schema and pagination.
- Harden join room validation and add TURN fallback for restricted networks.
- Preserve original contributors, original project links and authorized licensing terms in `CONTRIBUTORS.md`.

Migration sequence

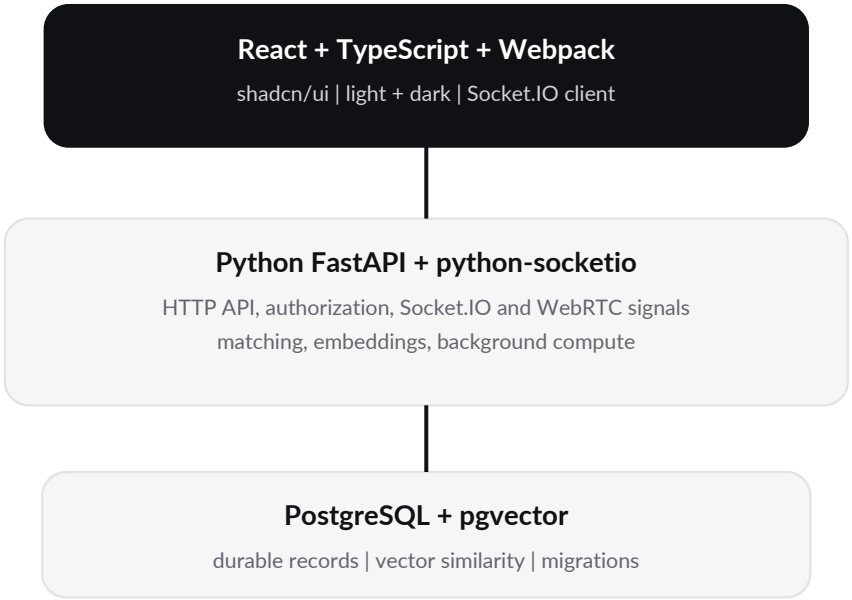
01. Obtain reuse permission. **02.** Inventory features and screenshots. **03.** Set up new React build. **04.** Implement FastAPI contracts and PostgreSQL migrations. **05.** Port chat and WebRTC signaling with tests. **06.** Add profiles, projects and matching.

No false continuity

The new Python server cannot execute the existing JavaScript controllers. Treat porting as new implementation, keep behavior tests, and mark the previous Express server as legacy reference code only.

03 Final system architecture

Use one frontend and one Python backend. Keep machine learning as internal Python modules rather than adding a second separately deployed API. Use PostgreSQL for relational state and pgvector for profile embeddings.



Application responsibilities

Boundary	Owns	Never trusts
React / Webpack	Rendering, forms, accessible navigation, local theme preference, realtime client.	Browser supplied roles or eligibility claims.
FastAPI	Sessions, permissions, domain operations, REST endpoints, signaling and socket events.	Client generated authorization decisions.
Python matching	Embeddings, candidate scoring, reasons, evaluation, controlled background jobs.	Unfiltered or blocked candidates as eligible.
PostgreSQL	Transactions, constraints, history, membership, consent, embeddings.	Unvalidated data from request handlers.

Architecture decisions

- **AD001:** React, TypeScript, Webpack 5 and shadcn/ui, with one shared design system.
- **AD002:** Python FastAPI is the only business backend; Python Socket.IO ASGI integration supports the existing Socket.IO browser client.
- **AD003:** SQLAlchemy 2, Alembic and PostgreSQL with pgvector replace Mongoose and MongoDB.
- **AD004:** The ML ranker runs inside the Python backend; evaluate moving it to a worker only when inference affects request latency.
- **AD005:** No Kubernetes, graph database, external vector database or autonomous agents in the initial release.

04 Industry standard repository structure

Use domain oriented modules, explicit contracts and only the infrastructure the application currently needs.

```
neu-connect-hub/
|-- apps/
|   |-- web/                        # React + TypeScript + Webpack
|   |   |-- public/                 # Favicons and static assets
|   |   |-- src/
|   |   |   |-- app/                 # Router, providers, error views
|   |   |   |-- features/            # auth, discovery, projects, chat, rooms, scan
|   |   |   |-- components/ui/       # shadcn generated primitives
|   |   |   |-- components/layout/   # Header, sidebar, mobile shell
|   |   |   |-- assets/brand/        # logo-light.svg, logo-dark.svg
|   |   |   |-- styles/globals.css   # Preset tokens + dark variables
|   |   |   `-- lib/                 # typed HTTP and Socket.IO clients
|   |   |-- webpack.config.cjs
|   |   |-- postcss.config.cjs
|   |   |-- components.json
|   |   |-- tsconfig.json
|   |   `-- package.json
|   `-- api/                        # Python backend: ONLY app server
|       |-- app/
|       |   |-- main.py             # FastAPI and ASGI composition
|       |   |-- core/               # Settings, security, errors
|       |   |-- api/v1/             # Routers and response schemas
|       |   |-- modules/            # auth, profiles, projects, chat, rooms, scan
|       |   |-- realtime/           # Socket.IO events, signaling
|       |   |-- ml/                 # embeddings, ranking, explanations
|       |   `-- db/                 # SQLAlchemy models and sessions
|       |-- alembic/                # Database migrations
|       |-- tests/                  # pytest: unit, API, Socket.IO
|       |-- experiments/            # Datasets, evaluation, Kaggle scripts
|       |-- pyproject.toml
|       `-- Dockerfile
|-- infra/                          # Compose, service configuration
|-- docs/                           # Decisions, security, API and ML
|-- scripts/                        # Synthetic seeds and smoke tests
|-- .github/workflows/               # Frontend and Python CI checks
|-- .env.example
|-- docker-compose.yml
|-- CONTRIBUTORS.md
`-- README.md
```

Code quality and dependency rules

- Use Python 3 type hints and Pydantic request/response schemas; enforce module boundaries with domain services and repositories where justified.
- Use `uv` or another lock capable Python dependency manager; preserve a committed lockfile and reproducible environments.
- Node.js and npm are needed locally and in CI for Webpack, shadcn CLI and frontend packages, **not for a production Node.js API server**.
- Keep frontend generated UI primitives in `components/ui`; feature components stay with their own feature.
- Run Ruff, mypy or Pyright, pytest, ESLint, TypeScript checks and React tests before merging changes.
- Use synthetic students for fixtures and demos, never copied classmates or private messages.

05 Design system, logo and light/dark themes

The application should follow the user selected shadcn preset. This PDF uses a restrained neutral design for readability; it does not invent the exact values encoded in an opaque preset. Use **Manrope** as the established typography requirement. Because the new preset has a different opaque code, verify its generated font configuration rather than assuming it contains the same font as the previous preset.

Approved preset command

Run from `apps/web` after React, Webpack, Tailwind, TypeScript alias resolution and `components.json` are configured. Preserve this exact user selected command:

```
cd apps/web
npx shadcn@latest apply --preset b27GdDLG
```

Important setup difference for a custom Webpack application

- shadcn does not provide an official starter template specifically for Webpack. Configure Tailwind CSS through PostCSS, CSS loaders and the correct global stylesheet import.
- Use the same `@/*` alias in `tsconfig.json`, `webpack.config.cjs` and `components.json`. Set React Server Components to false.
- Initialize the existing React project with the shadcn CLI or the official manual setup first; verify generated files and then apply the preset on a separate branch.
- The apply operation can replace configuration, CSS variables, fonts and installed components. Commit current changes first, review the generated diff and verify both color themes.
- Use the preset generated design tokens rather than guessed colors or duplicated hardcoded component styles.
- After applying the new preset, inspect the generated font settings. Keep Manrope in both themes as previously requested, configuring it once in the design system if the new preset selects a different font. Avoid duplicate font imports or conflicting overrides.

Approved logo variants



Light surface | Dark mark



Dark surface | White mark

The mark above is reproduced from the supplied image. Keep aspect ratio, padding and shape unchanged. Export transparent dark and light variants as optimized SVG or PNG after inspecting the final asset; the white version is for dark surfaces and the dark version for light surfaces.

Theme implementation contract

Concern	Requirement
Theme values	Support light, dark and system preferences; persist explicit choice in local storage.
Initialization	Apply theme class before initial paint to avoid flashing the wrong theme.
Brand mark	Select the light surface mark or dark surface mark based on active theme.
Colors	Use preset driven CSS variables such as background, foreground, border and muted.
Accessibility	Respect contrast, focus, keyboard operation, reduced motion and user color preferences.
Release review	Test onboarding, matching, project forms, messaging and video UI in both modes.

06 Relational database and migrations

Adopt PostgreSQL with SQLAlchemy and Alembic. pgvector stores profile embeddings alongside canonical student records, avoiding a second vector service. Every relationship must be backed by foreign keys, unique constraints and authorization tests.

Table	Representative fields	Constraints / indexes
users	id, email, password_hash, verified_at, created_at	unique normalized email; no raw passwords
profiles	user_id, headline, bio, discoverable, updated_at	unique user_id; opt in visibility
skills / profile_skills	skill_id, label, user_id, proficiency	unique user skill pair; controlled taxonomy
interests / availability	user_id, topic, weekday, start, end, timezone	valid time ranges and normalized categories
projects	id, owner_id, title, summary, capacity, status	owner foreign key and status index
applications	project_id, applicant_id, status	unique project/applicant pair
connections	sender_id, recipient_id, state	unique active pair; recipient authorization
profile_share_tokens	user_id, token_hash, active, created_at	non enumerable token; rotation and revocation
conversations / members	id, kind, user_id	membership required for private reads
messages	conversation_id, author_id, client_msg_id, body	unique author/client id; time index
rooms / room_members	id, project_id, user_id, room_state	authorize join and signaling
embeddings	user_id, model_version, embedding, updated_at	vector dimension checked; purge on deletion
match_feedback	viewer_id, candidate_id, action, consent	minimize collection; policy controlled

Integrity and privacy invariants

- Only verified and opted in students appear in the discovery pool; blocked users are excluded before ranking.

- Accepting an invitation must check the recipient and work correctly on retries and concurrent requests.
- Project team acceptance uses a transaction with membership and capacity checks.
- Private messages are available only to active conversation members; membership is verified on every request and socket event.
- Embeddings are versioned derivatives of opted in fields; deletion removes embeddings and downstream feedback according to the retention policy.

Migration workflow

```
alembic revision --autogenerate -m "create core tables"
alembic upgrade head
python -m scripts.seed_demo   # define this module during setup
```

07 FastAPI contracts and Python realtime

Public HTTP API proposal

Route	Purpose	Authorization
POST /api/v1/auth/register	Create account and issue verification challenge.	Public + abuse protection
POST /api/v1/auth/login	Start secure session and set cookies.	Public + throttling
GET /api/v1/me	Read current profile and verification state.	Authenticated
PUT /api/v1/me/profile	Manage goals, skills, availability, visibility.	Account owner
GET /api/v1/discovery	Eligible and ranked peer recommendations.	Verified and opted in
POST /api/v1/connections	Request collaborator connection.	Verified student
POST /api/v1/connections/{id}/accept	Accept addressed invitation.	Intended recipient
GET /api/v1/me/connection-code	Get personal QR profile link.	Account owner
POST /api/v1/me/connection-code/rotate	Rotate personal QR profile link.	Account owner
GET /api/v1/connect/resolve/{token}	Resolve approved profile preview.	Authenticated
POST /api/v1/projects	Create project opportunity.	Verified student
POST /api/v1/projects/{id}/apply	Apply to project.	Eligible student
GET /api/v1/conversations/{id}/messages	Cursor based private message history.	Member
POST /api/v1/rooms	Create protected room.	Permitted member

Socket.IO and signaling without Node.js

Use **python-socketio.AsyncServer** with **socketio.ASGIApp** wrapping the FastAPI application. Retain the JavaScript `socket.io-client` in React. Browser media remains WebRTC; Python only relays authorized signaling metadata.

```
# apps/api/app/main.py (conceptual composition)
from fastapi import FastAPI
import socketio

http_app = FastAPI()
sio = socketio.AsyncServer(async_mode="asgi", cors_allowed_origins=[])
app = socketio.ASGIApp(sio, other_asgi_app=http_app)

# Register routers, authentication checks and socket handlers.
# Configure explicit allowed browser origins before deployment.
```

Event contracts

Event	Server responsibility
chat.send	Authenticate membership, persist in PostgreSQL, acknowledge idempotent message ID.
chat.message	Deliver to authorized members; permit history sync after reconnect.
presence.changed	Scope visibility to permitted viewers.
room.join	Validate room ID, room permissions and capacity before signaling.
rtc.signal	Relay only within the authorized room.
room.leave	Remove member presence and notify the remaining authorized peers.

Concurrency and deployment note

Start with one ASGI worker and test connection behavior. In memory room state and simple Socket.IO broadcasting are not sufficient for multiple workers. Add a shared manager such as Redis only when scaling beyond one instance, and keep room authorization in PostgreSQL. Avoid CPU intensive embedding inference on the event loop.

08 Machine learning recommendations

Eligibility before ranking

```
1. Identify requester from authenticated session.
2. Filter to verified, opted in and visible student profiles.
3. Exclude requester, blocked accounts and ineligible candidates.
4. Load updated profile embeddings, skills and availability.
5. Score and rank; return five candidates with grounded reasons.
6. Log permitted minimum exposure data, if consented.
```

Transparent initial scoring baseline

score = 0.35 × semantic interest + 0.30 × complementary skills + 0.25 × availability overlap + 0.10 × goal alignment

Each input is scaled to [0, 1]. The weights are hypotheses, not trained probabilities. Validate with explicit labels and real feedback before claiming accuracy improvements.

Component	Implementation
Semantic interest	Sentence Transformers embeddings from selected public profile fields, compared by cosine similarity.
Complementary skills	Cover missing required skills for the requester project, not merely identical keywords.
Availability overlap	Use explicit time zone aware weekly slot intersection.
Goal alignment	Differentiate studying, project development, research and mentorship.

Python implementation guidance

- Keep the ranker in `apps/api/app/ml`, with isolated Python modules for embeddings, filters, scoring and explainability.
- Compute profile embeddings asynchronously when profiles change. The API should read the latest accepted embeddings, not generate them on every request.
- Preselect and permission filter in PostgreSQL before vector scoring; add approximate indexes only if measured candidate volume requires them.
- When models fail, fall back to deterministic rules and record an internal reason code; never bypass eligibility checks.
- Do not feed private chat, call transcripts or inferred sensitive traits into the matching model.

Complementary team composition

Use a greedy coverage baseline: from willing, eligible candidates, select those who maximize remaining required skill coverage while meeting availability and capacity constraints. Suggest a team; never automatically enroll students.

09 Machine learning evaluation and Kaggle

One reproducible experiment is better than many vague AI features. Keep the Kaggle social graph benchmark separate from user facing campus recommendation metrics.

Experiment	Data source	Evaluation
Profile matching	Synthetic profiles and carefully labeled candidate pairs.	Precision at 5 and normalized discounted cumulative gain at 5.
Graph link prediction	Kaggle Facebook Recruiting anonymized graph.	Mean average precision at 10 with proper graph splits.
Learned ranking (later)	Only sufficiently representative, permissioned campus feedback.	Time split and comparison to transparent baseline.

Reproducibility checklist

- Document consent, dataset source, label definitions, split strategy, seeds, hyperparameters and model version.
- Compare rules only, embeddings plus rules and a learned ranker only if sufficient data exists.
- Avoid using future interaction edges to build training graph features for earlier prediction targets.
- Report actual measured ranking performance, inference latency and failure rate. Do not invent metrics or display model scores as calibrated success probabilities.
- Prefer scikit learn and LightGBM only where the evaluation demonstrates a real benefit.

10 Security, privacy and operational trust

Area	Required controls
Secrets	Do not commit real credentials; rotate compromised legacy secrets. Use secret scanning.
Authentication	Verify campus email ownership, secure cookies and appropriate cross site request protections.
Authorization	Enforce recipient, project, membership and room permissions on Python HTTP and Socket.IO handlers.
Schema validation	Use Pydantic constraints, safe serialization and bounded request sizes.
Realtime transport	Restrict origins, authenticate socket sessions and authorize every room event.
Video	Issue short lived TURN credentials; never persist raw meeting media by default.
Privacy	Discovery opt in, block/report features, retention policy, deletion of derived embeddings.
Auditing	Structured request identifiers and sanitized errors; never log secrets or private message bodies.

Required negative tests

- An unrelated student cannot accept an invitation addressed to someone else.
- A nonmember cannot read another conversation or send to another project room.
- A blocked user is excluded from discovery even if the ML model scores them highly.
- Expired verification links fail; duplicate message sends are idempotent.
- Invalid socket sessions or disallowed origins cannot access protected events.
- Unavailable or malformed room IDs return safe errors instead of crashing the ASGI server.

11 Testing, release automation and deployment

Test and verification matrix

Layer	Tooling	Required checks
Python unit	pytest	Ranking, constraints, permissions, time zone overlap.
Python integration	pytest + HTTPX + test PostgreSQL	Auth, projects, database transactions and private messaging.
Socket integration	python-socketio client / browser test	Authentication, event acknowledgments, reconnections and room membership.
React unit	Vitest or Jest + Testing Library	Theme toggle, forms, accessibility states and match cards.
Browser acceptance	Playwright	Onboarding to discovery to invitation to chat and room.
Performance	Locust or k6	Warm ranking latency, API errors and video signaling reliability.

Continuous integration workflow

```
On pull requests:
  npm ci                          # React / Webpack tools only
  npm run lint && npm run typecheck && npm run build
  python -m pip install -e "apps/api[dev]"
  ruff check apps/api && pytest apps/api/tests
  alembic check                   # after migrations are configured
  dependency and secret scans

For release:
  deploy static frontend and Python ASGI API
  migrate PostgreSQL before serving traffic
  smoke test health, auth, sockets and both themes
```

Deployment topology

- Serve static Webpack assets from a static host or reverse proxy. No Node.js application server is needed.
- Deploy one Python ASGI service with Uvicorn or an appropriate ASGI process manager; scale only after verified load testing.
- Use managed PostgreSQL with pgvector enabled, migrations, backups and scoped database permissions.
- Provide a TURN solution for WebRTC. LiveKit or a similar media server is a later option only if group call reliability requires it.
- Store runtime secrets in the deployment platform; configure domain origins, TLS, health checks and safe rollback.

12 Environment and development workflow

Configuration examples

```
# apps/api/.env.example
DATABASE_URL=postgresql+asyncpg://app:local@localhost:5432/nu_hub
SESSION_SECRET=replace_with_random_local_secret
WEB_ORIGIN=http://localhost:8080
EMAIL_PROVIDER_API_KEY=replace_if_needed
EMBEDDING_MODEL_ID=sentence-transformers/all-MiniLM-L6-v2

# apps/web/.env.example (Webpack injection, not VITE_* vars)
API_BASE_URL=http://localhost:8000/api/v1
SOCKET_URL=http://localhost:8000
```

Do not expose secret backend configuration through Webpack DefinePlugin or injected browser variables. The model identifier above is illustrative and should be evaluated for cost, licensing and relevance.

Intended developer commands

```
git clone https://github.com/kshitijkumar-northestern/neu-connect-hub.git
cd neu-connect-hub

docker compose up -d postgres
cd apps/api
python -m venv .venv
# Activate virtual environment for your operating system.
python -m pip install -e ".[dev]"
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# A separate shell for the React / Webpack frontend:
cd apps/web
npm ci
npm run dev
npx shadcn@latest apply --preset b27GdDLG
```

These commands define the implementation target. They are not verified against a scaffolded repository yet. Add the scripts, package configuration and environment templates in Phase 1 and revise documentation to match tested behavior.

Minimal Webpack and shadcn configuration checklist

- Webpack 5 configured to compile TSX with an appropriate Babel or TypeScript loader, resolve the @ alias and import CSS.
- Tailwind CSS processed with `@tailwindcss/postcss` and PostCSS; load the preset CSS from the frontend entrypoint.
- Use `components.json` aliases that exactly match Webpack and TypeScript.
- Define a local theme provider that supports system, light and dark, and swaps logo treatment accordingly.
- Use consistent versioned browser API base configuration for development and production.

13 Delivery phases and recruiter proof

Phase	Work	Acceptance criteria
01 Foundation	Provenance, secrets, Webpack React, FastAPI, PostgreSQL schema.	Two test users can register and sign in; CI runs both stacks.
02 Collaboration	Port messaging and WebRTC signaling, projects, profiles, permissions.	Two users can message and join permitted room; negative tests pass.
03 Intelligence	Python embeddings, weighted ranker, team coverage, explanations.	Five eligible recommendations with evidence and fallback.
04 Evaluation	Kaggle benchmark, annotations, quality and latency comparison.	Repeatable report without data leakage or invented results.
05 Release	Accessibility, light/dark screenshots, deploy, docs and demo.	Fresh checkout reproduces tests and user journey.

First eleven GitHub issues

Issue	Definition of done
01. Confirm source rights	Original team source and reuse decisions documented.
02. Eliminate tracked secrets	Rotations and environment templates implemented.
03. Scaffold React + Webpack	TypeScript build, Tailwind and aliases working.
04. Apply shadcn preset	Preset b27GdDLG, Manrope typography and both logo/theme modes reviewed.
05. Scaffold FastAPI + PostgreSQL	Python API starts; Alembic migrations pass.
06. Port auth and profiles	Verified student opt in and ownership tests pass.
07. Port Socket.IO and calls	Messages durable; private room signaling secured.
08. Projects and teams	Applications, capacity rules and approvals implemented.
09. Scan to Connect	Personal QR share, scan, preview, revocation and auth tests.
10. Recommendations	Ranker with grounded reasons and ML fallback.
11. CI, benchmark and demo	Metrics, synthetic user journey and screenshot proof.

Resume and interview proof

Demonstrate one meaningful product journey, the Python architecture, tests for security regressions and a reproducible ML experiment. Attribute historical team contributions precisely. Replace all hypothetical claims with verified outcomes from actual tests, commits or benchmark reports.

14 Scan to Connect: personal QR profiles

Implement **Scan to Connect** as a fast, consent based path to peer connection. A QR code is the appropriate machine readable barcode for sharing a profile URL; a retail UPC or EAN code is not suitable for this feature.

User journey and screens

```
Owner: Profile > My Code > display or share QR link
Peer: Discover > Scan > camera permission > QR detected
     > authenticated profile preview > Connect request
Owner: accept or decline invitation > messaging enabled
```

- A personal QR code displays the profile entry link but no email, session token, private contact information or private project details.
- A scanner is available in discovery. Include a paste link fallback and a file upload option where camera access is unavailable.
- After scanning, show a limited profile preview and an explicit Send connection request action. Do not auto connect or auto open private chat.
- The owner can regenerate or disable their public scan token. Blocked or hidden profiles cannot be resolved for an unauthorized viewer.
- Use identical shadcn form, card, dialog and toast components in light and dark modes. Verify the final Manrope setting after applying the new preset.

Frontend design and implementation

Path / module	Implementation requirement
features/scan/components/MyQRCode.tsx	Render personal QR with a lightweight QR generator; copy and share actions.
features/scan/components/ScanCodeDialog.tsx	Request camera permission only on user action; scan once, stop media tracks when closed.
features/scan/pages/ConnectPreview.tsx	Fetch a minimal authorized profile preview and initiate an invitation.
features/scan/lib/qr.ts	Validate URL origin, parse route/token; avoid accepting arbitrary scanner redirects.

Backend contracts and persistence

Contract	Requirement
GET /api/v1/me/connection-code	Create or return current owner share link. Return URL and optional expiry.
POST /api/v1/me/connection-code/rotate	Revoke previous code and create a new cryptographically random share token.
GET /api/v1/connect/resolve/{token}	Require active session; validate token, opt in visibility and blocks. Only return approved preview fields.
POST /api/v1/connections	Existing connection invitation endpoint; authorize target and prevent self or duplicate requests.
profile_share_tokens	Persist only a token hash; bind to owner and revocation state; index lookup and audit rotation.

Security, analytics and acceptance tests

- Use opaque, high entropy random tokens rather than sequential student IDs, signed in links or login credentials. Enforce HTTPS and do not log raw tokens.
- Never auto join a room on scan. Resolve the profile through server authorization, then require an invitation acceptance to connect.
- Rate limit token resolution and invitation creation. Prevent scanning code links from bypassing visibility, verification or block policies.
- Test invalid, revoked, expired and malformed tokens; camera denial; double scans; dark theme contrast; keyboard interaction and connection request deduplication.
- Report meaningful metrics only after implementation, such as scan completion and invitation acceptance, without exposing scan identities in analytics.

Release acceptance gate

Two distinct test users can display, scan and resolve a profile QR, send and accept an invitation, and subsequently message. A blocked profile, disabled code and invalid token must all fail without disclosing private data.

15 Risks, architectural decisions and sources

Risk	Mitigation
Python migration is larger than a refactor	Keep behavior contracts and port tested, approved functionality incrementally.
Realtime scaling complexity	Run a single ASGI worker initially; introduce a shared manager only with load evidence.
Small dataset for ML	Use transparent ranking; reserve learned models for real data and correct evaluation.
Custom Webpack with shadcn	Manual Tailwind and alias setup before applying preset; review the generated diff.
Dark mode produces unreadable screens	Test themes with real flows, contrast checks and logo variants.
QR code exposes private identity	Use opaque revocable tokens, approved preview fields, authenticated resolve and explicit consent.
Old repository provenance unclear	Obtain permission or reimplement modules; document contributors.

Official documentation and reference sources

FastAPI official documentation

Python Socket.IO ASGI server integration

SQLAlchemy 2.0 documentation

Alembic migrations

pgvector Python SQLAlchemy integration

Webpack official guides

shadcn CLI preset apply

shadcn official manual installation

Sentence Transformers

OWASP Application Security Verification Standard

Kaggle Facebook Recruiting

Original ConnectRoom repository

Original NUConnects repository

Target repository

Recommended execution gate

Start with architecture and theme foundations

Begin by confirming source reuse rights, scaffolding React with Webpack and FastAPI with PostgreSQL, and configuring preset driven light/dark shadcn styles. Add Scan to Connect only after Python authentication and permissions work.